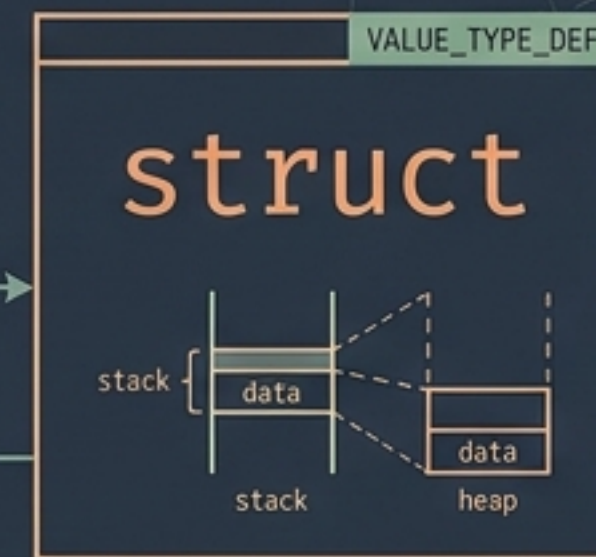
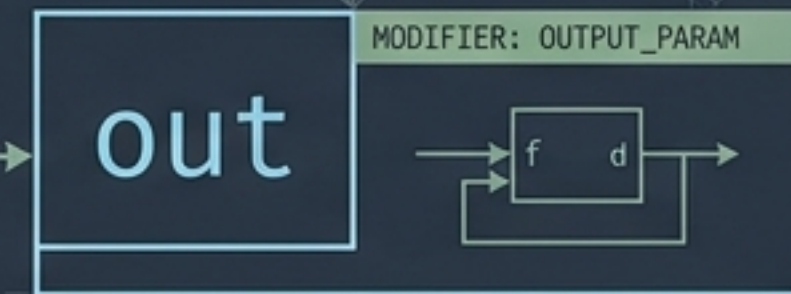
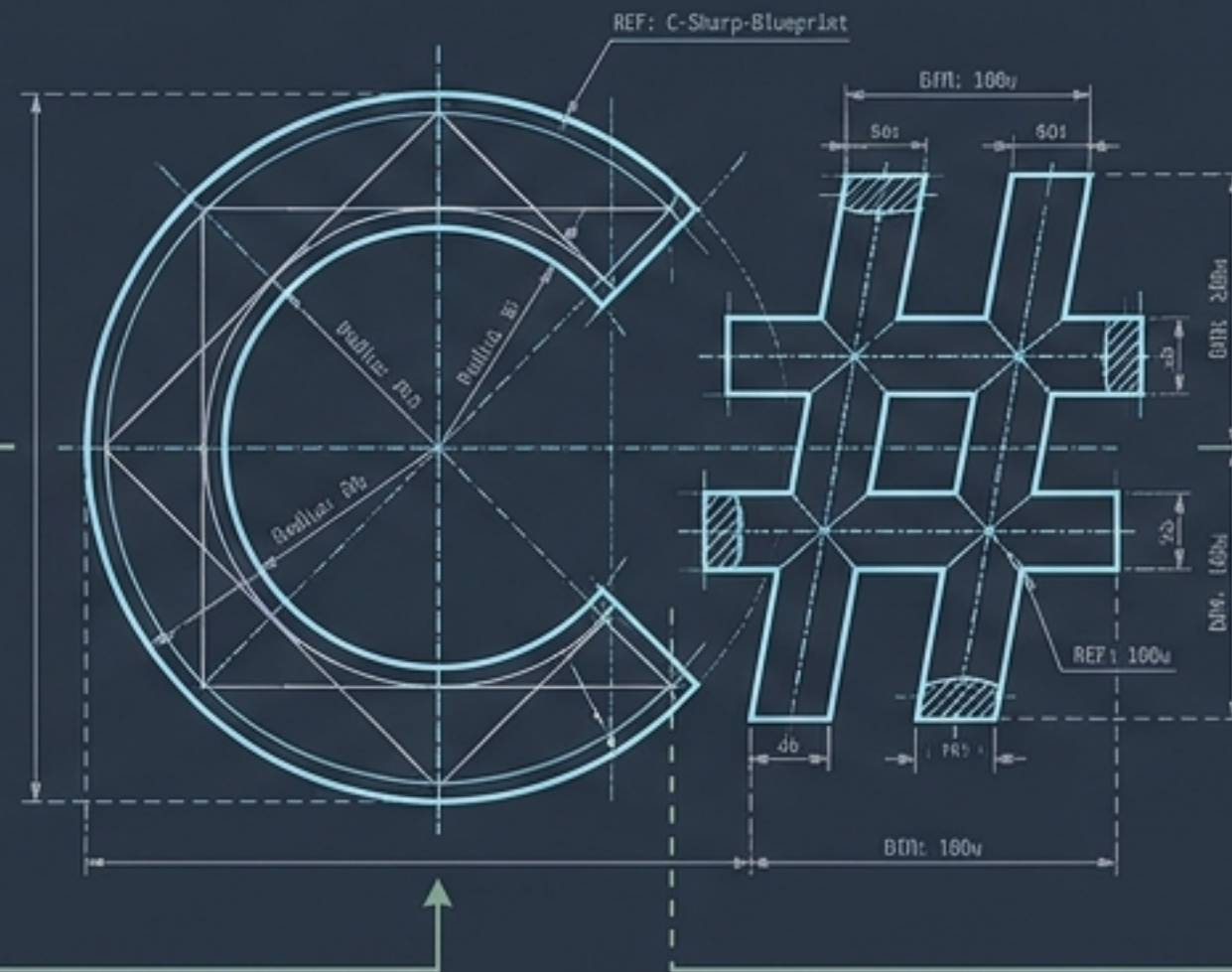
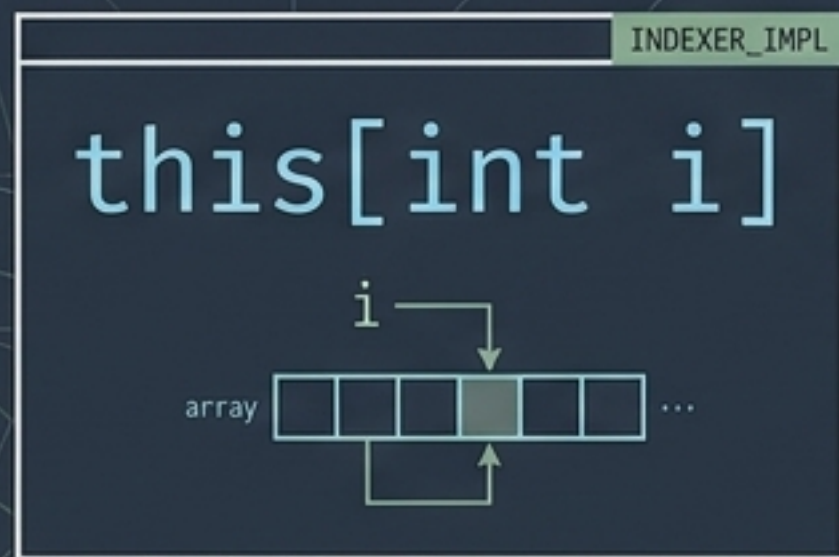
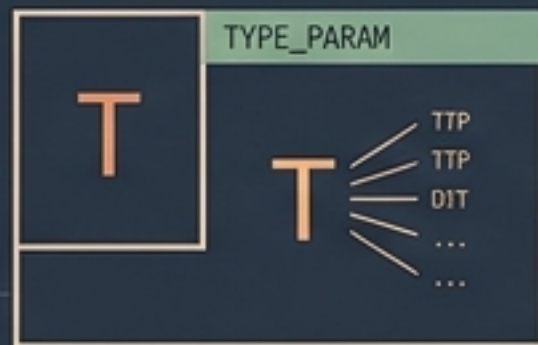


C# 클래스 심화: 더 강력하고 유연한 코드 설계

제네릭, 인덱서, Out, 구조체 마스터하기 (Visual Technical Guide)





모듈 1 (자료형의 자유): 제네릭 (Generics)

타입에 얽매이지 않는
유연한 **템플릿** 설계.



모듈 2 (배열처럼 다루기): 인덱서 (Indexers)

객체를 직관적으로 **순회**하고
접근하는 기술.



모듈 3 (단일 반환의 한 계 돌파): **out** 키워드

하나의 메서드에서 여러
결과값을 뱉어내는 **파이프라인**.



모듈 4 (가볍고 단단한 데이터): 구조체 구조체 (Structs)

무거운 클래스를 대체하는
빠르고 안전한 **값(Value)** 캡슐.

제네릭(Generics)의 해부학: 자료형의 자유

class Wanted

<T>

클래스 선언 시 사용하는 꺾은 괄호.
내부의 데이터 타입을 외부에서
결정하도록 엽니다.

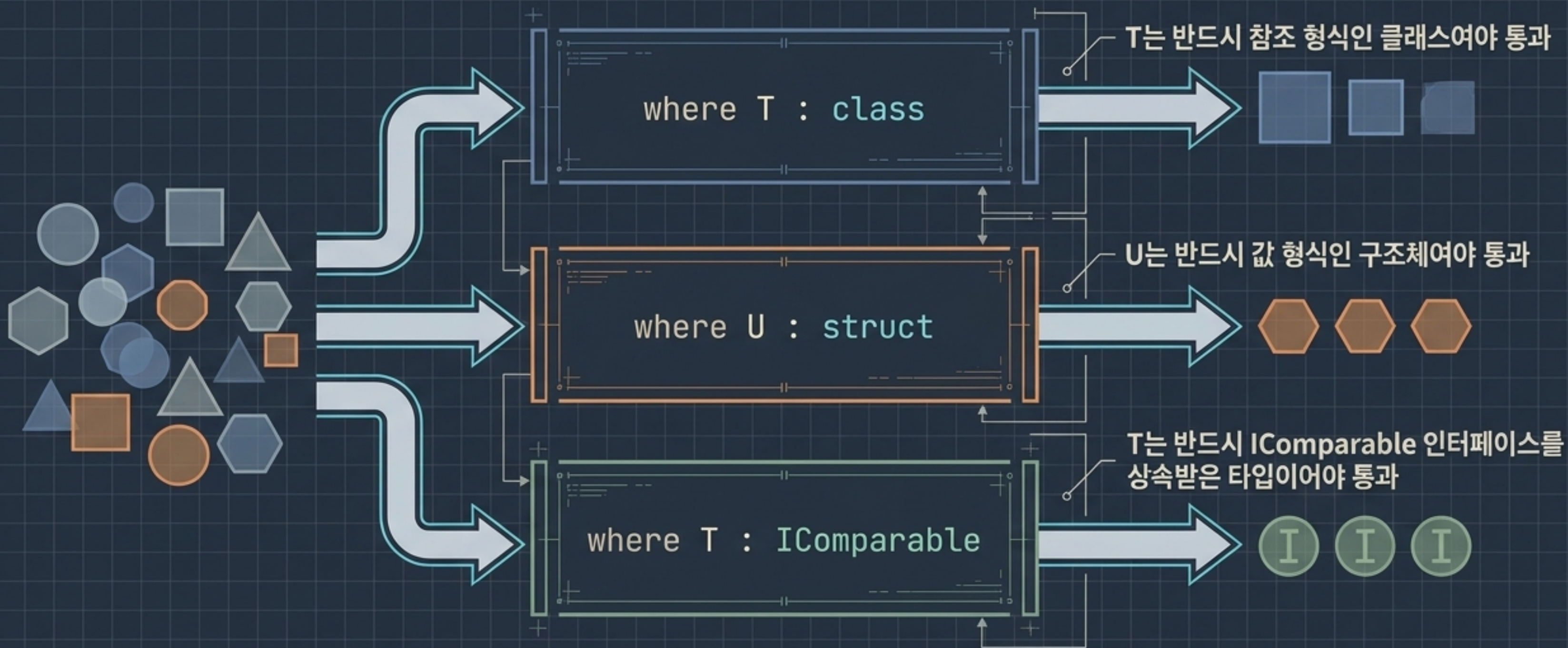
식별자(Alias). int, string, float 등 어떤
자료형이든 들어올 수 있는 '마법의
빈칸'입니다.

타입의 무한한 변신 (Flexibility in Action)

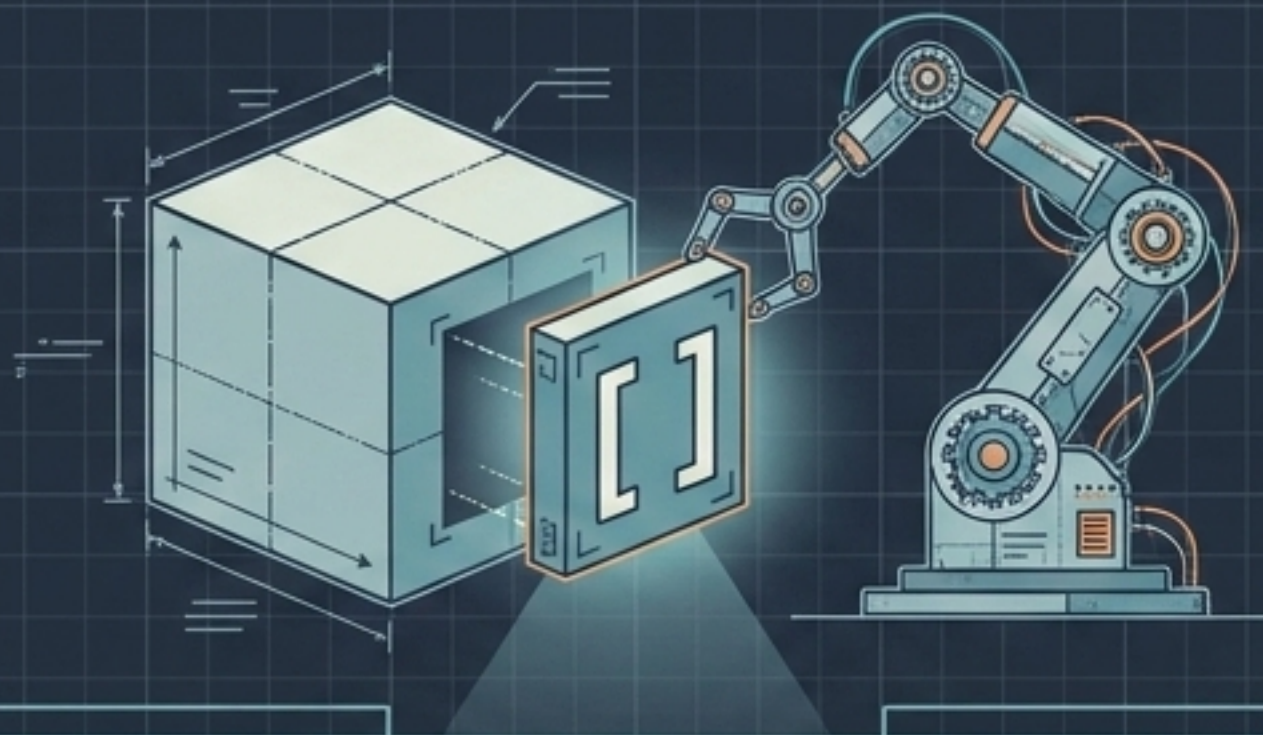


두 개 이상의 제네릭이 필요하다면? `<T, U>`처럼 심표로 확장할 수 있습니다.

제네릭의 안전장치: where 제약 조건



인덱서(Indexers): 객체를 배열처럼 직관적으로



this: 인덱서를 선언하는 예약어.
객체 자신을 지칭합니다.

```
public int this[int i] {  
    get { ... }  
    set { ... }  
}
```

[int i]: 배열처럼 괄호 안에 들어갈
인덱스의 자료형을 정의합니다.

get / set: products[i]를 호출할 때(get), 또는
products[i] = 10처럼 값을 할당할 때(set) 실행되는 블록입니다.

인덱서 작동 원리: SquareCalculator

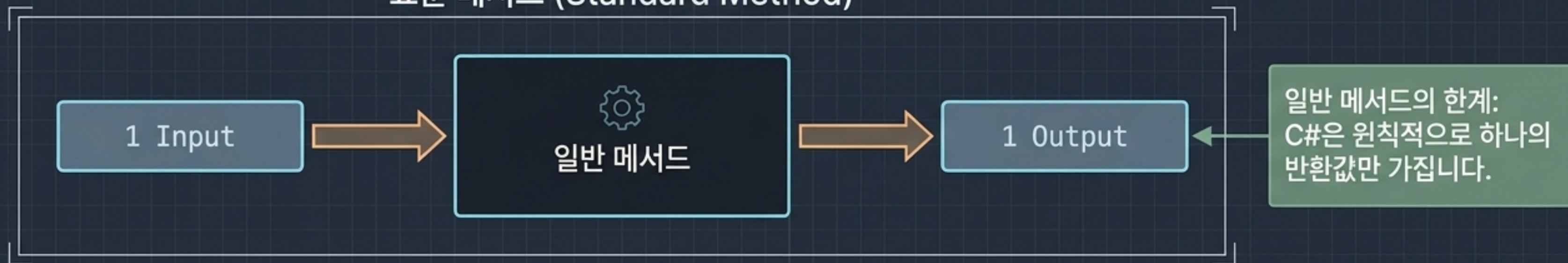


`square[10]` → `return 10 * 10;` → 결과: 100

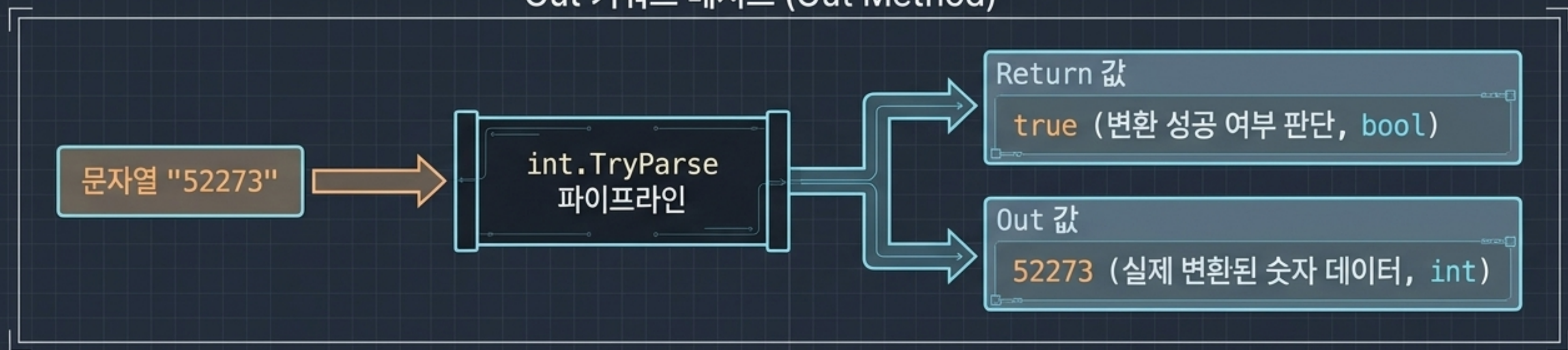
배열이 아닌 사용자 정의 클래스도 인덱서를 통해 내부에 복잡한 연산 로직을 숨기고 직관적인 데이터 접근을 제공할 수 있습니다.

다중 반환의 한계 돌파: out 키워드

표준 메서드 (Standard Method)



Out 키워드 메서드 (Out Method)



out 키워드 실전 문법 (NextPosition)

```
static void NextPosition(...,  
    out int rx, out int ry)
```

```
NextPosition(..., out x, out y);
```

주의사항: 변수가 아닌 일반 자료(상수 등)를 넣거나, 호출 시 out 키워드를 누락하면 오류가 발생합니다.
매개변수로 넣은 변수에 직접 값을 '채워' 넣는 방식입니다.

구조체(Struct): 가볍고 간단한 데이터 캡슐



Class: 무겁고 다루기 복잡하지만 큰 수용력



Struct: 가볍고 빠르며 간단함

- **개념:** 간단한 객체를 만들 때 사용하는 형식으로, 클래스와 문법은 유사하지만 더 제한적이고 안정성에 초점이 맞춰져 있습니다.
- **비밀:** 우리가 흔히 쓰는 C#의 기본 자료형(**int**, **float** 등)은 사실 모두 내부에 구조체로 정의되어 있습니다.

깔깔한 구조체의 초기화 규칙



매개변수 없는 생성자 명시적 선언 불가 (자동 제공됨).

```
public Point() {  
    ...  
}
```



선언과 동시에 멤버 변수 초기화 불가
(예: `public string testB = "init";`)

```
public string testB = "init";
```



생성자 내부에서 멤버 일부만 초기화
(구조체 안의 클래스 인스턴스 포함, 모든 변수를
초기화해야 에러 발생 안 함)

```
public Point() {  
    testB = "init";  
    testB2 = 0;  
}
```



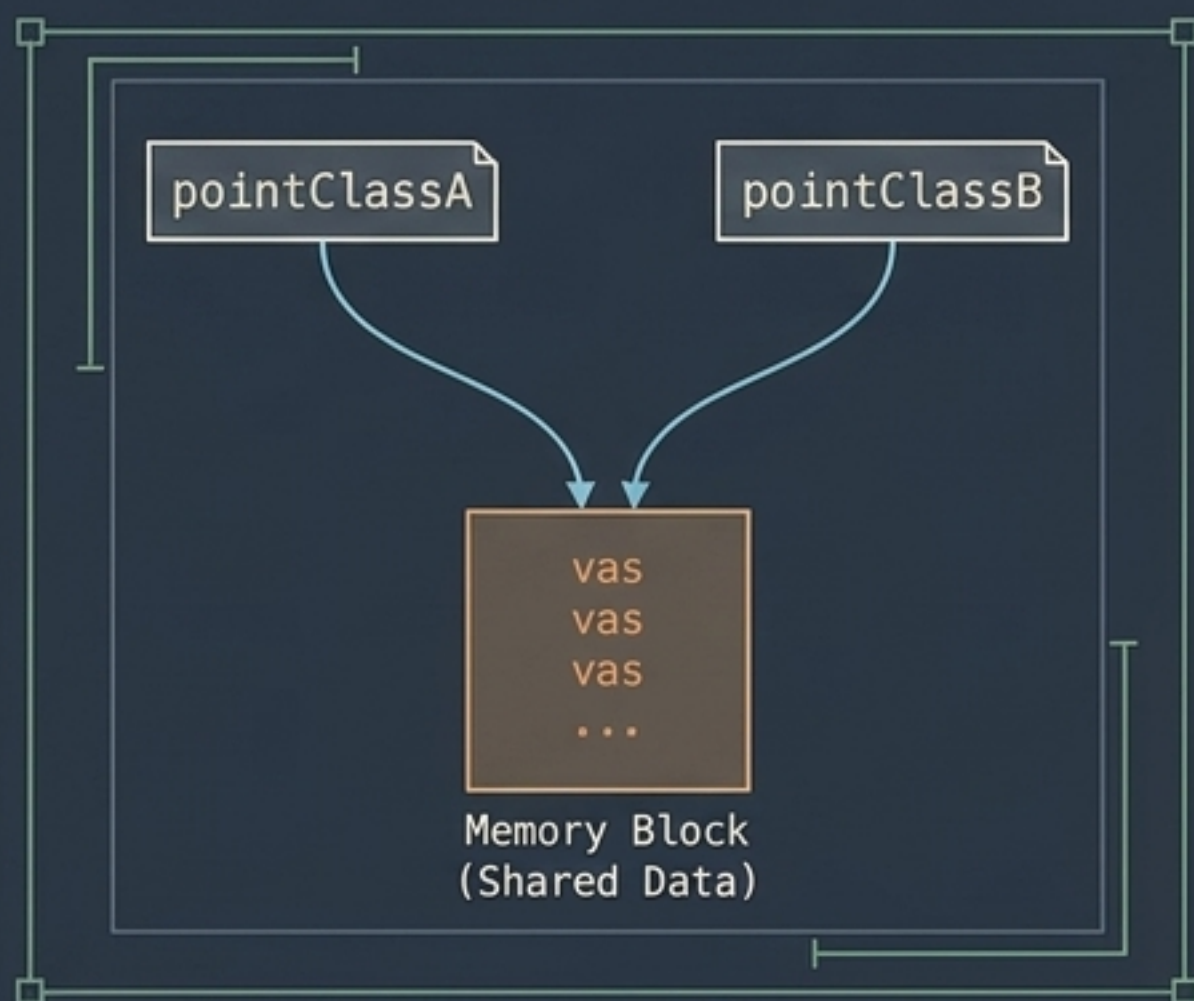
모든 변수를 초기화하는 매개변수 있는 생성자 작성

```
public Point( string string ) {  
    testA = init;  
    testB = 0;  
    testB = "init";  
}
```

클래스 vs 구조체의 결정적 차이: 메모리 복사 방식

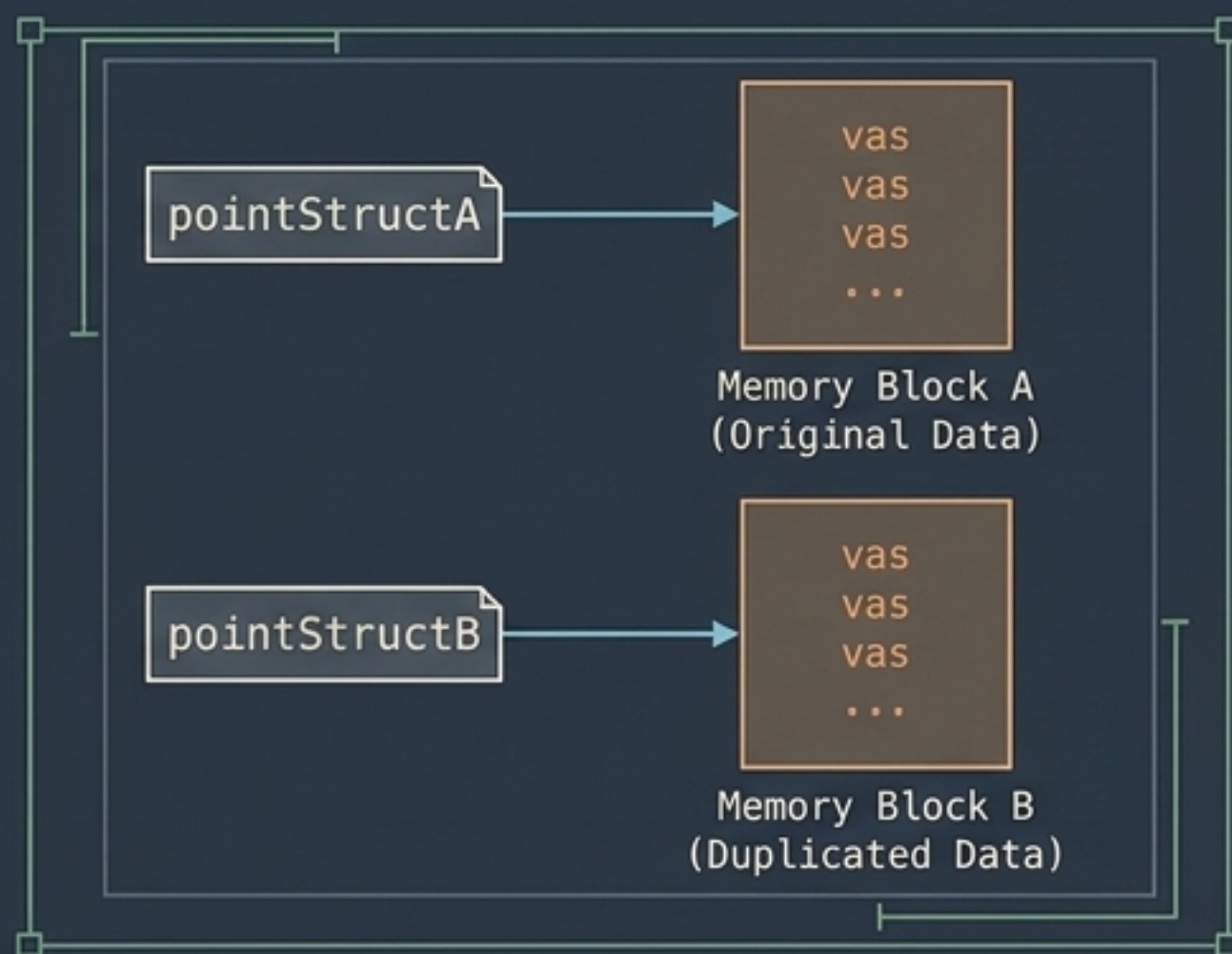
참조 복사 (Reference Copy)

`pointClassB = pointClassA`



값 복사 (Value Copy)

`pointStructB = pointStructA`



결과적 차이: 클래스는 하나의 값을 바꾸면 연결된 다른 변수의 값도 변하지만, 구조체는 원본과 복사본이 완전히 독립적으로 동작하므로 데이터 오염을 막을 수 있습니다.

의사결정 매트릭스: Class or Struct?

기준 (Criteria)	클래스 (Class)	구조체 (Struct)
복사 방식	참조 복사 (Reference)	값 복사 (Value)
초기화 규칙	자유로움	엄격함 (모든 멤버 강제 초기화)
가벼움/속도	무거움 (Heap 메모리)	가볍고 빠름 (Stack 메모리)
권장 사용처	상속이 필요하고 수명이 긴 복잡한 객체	수학적 좌표, 크기가 작고 단순한 데이터 컨테이너

Chapter 08 핵심 문법 마스터 시트 (Cheat Sheet)

Generics

```
class Wanted<T>
{
    public T Value;
}
```

Indexers

```
public int this[int i] {
    get { ... }
    set { ... }
}
```

Out Keyword

```
int.TryParse("123", out int result);
```

Structs

```
struct Point {
    public int x;
    public Point(int x) {
        this.x = x;
    }
}
```